



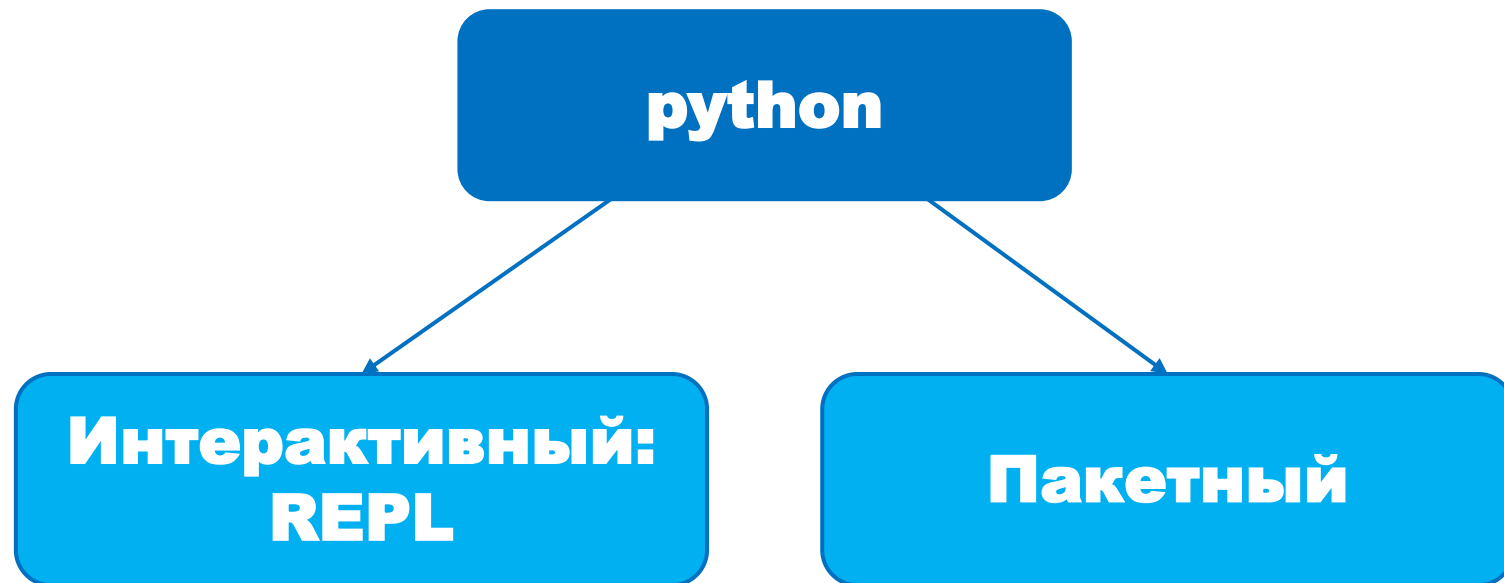
Анатомия Python-программ

Докладчик: Евграфов Михаил

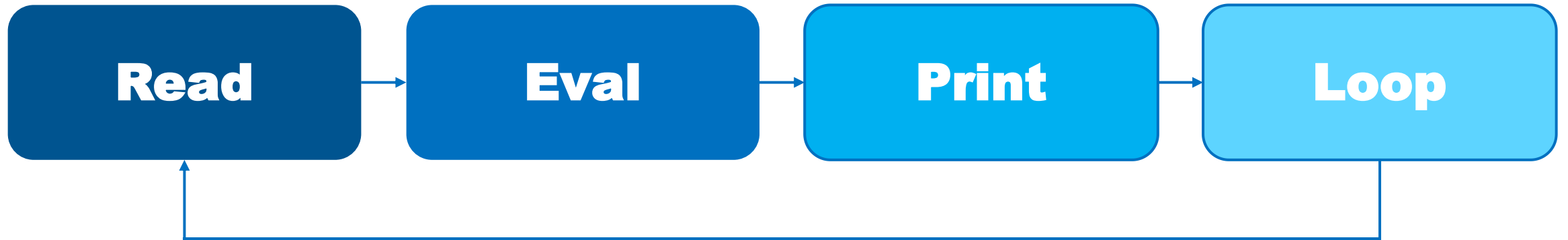


Запуск программ на Python

Режимы работы интерпретатора



REPL: Read – Eval – Print – Loop



Что умеет REPL

вывод без print()

```
>>> 2 + 3
5
```

переменная _

```
>>> 10 * 5
50
>>> _ + 1
51
```

многострочный ввод

```
>>> if 3 > 2:
...     print("greater")
...
greater
```

help() / dir()

```
>>> help(len)
Help on built-in function len in module builtins:

len(obj, /)
    Return the number of items in a container.
```

REPL: плюсы и минусы

Хорош для

эксперименты

проверка поведения

изучение объектов

Не годится для

большие программы

повторный запуск

командная работа

Пакетный режим

REPL

```
>>> 2 + 3  
5
```

Скрипт .py

```
# содержимое five.py  
print(2 + 3)
```

```
# вызов в терминале  
$ python add.py  
5
```

Полезные способы запуска

Команда	Что делает
<code>python script.py</code>	выполнить файл-скрипт
<code>python</code>	запустить REPL
<code>python -c "print(2 + 3)"</code>	код прямо из командной строки
<code>python -m http.server</code>	запустить модуль как программу
<code>python -i script.py</code>	выполнить и остаться в REPL

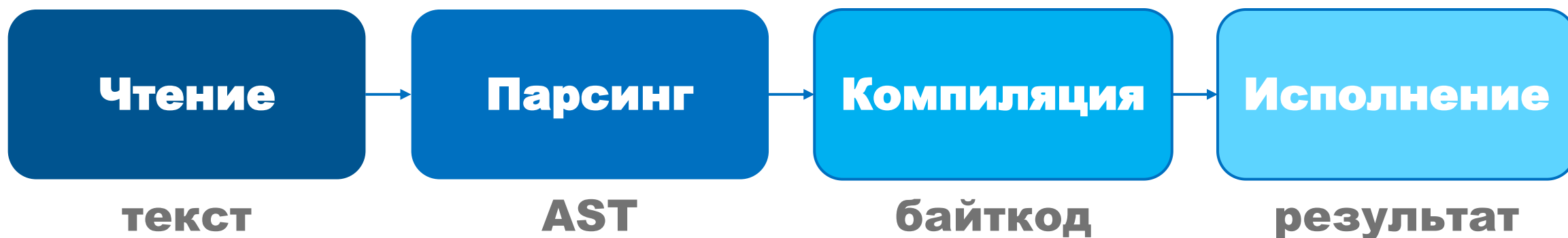
.пус и __pycache__





Как интерпретатор запускает программу

Конвейер: от текста к результату



Парсинг: Лексический анализ

- лексер разбивает текст на токены
- пробелы отбрасываются
- отступы → **INDENT / DEDENT**

```
# исходный код  
total = price * 2
```

```
# токенизация  
NAME 'total'  
OP '='  
NAME 'price'  
OP '*'  
NUMBER '2'  
NEWLINE
```

Парсинг: Синтаксический анализ

- парсер проверяет грамматику
- строит дерево AST
- ошибка → `SyntaxError`

Компиляция в байткод

Исходный код

```
def total(price):  
    return price * 2
```

компиляция



Байткод

```
LOAD_FAST      price  
LOAD_CONST     2  
BINARY_OP      *  
RETURN_VALUE
```

Компилируемый или интерпретируемый

- **компилируется в байткод (не машинный код)**
- **интерпретируется: PVM исполняет байткод**
- **в C сразу машинный код, без VM**



Анатомия Python-файла

Строки

физическая строка

```
light_velocity = 3e8
```

логическая строка

```
numbers = [  
    1, 2, 3, 4,  
    5, 6, 7, 8,  
]
```

Комментарии

вся эта строка является комментарием

distance_max = 72 * 1852 # эта часть строки - комментарий

Использование комментариев

правильное использование комментариев

```
# максимальная дальность обзора РЛС в морских милях  
# согласно протоколу использования РЛС, с.3 таб.1  
distance_max = 72 * 1852
```

неправильное использование комментариев

```
distance_max = 72 * 1852 # определили переменную
```

Пустые строки

```
if number % 2 == 0:
    number_type = "even"
else:
    number_type = "odd"
print(f"I got {number_type} {number}")
```



- первый логический блок кода
- второй логический блок кода
- третий логический блок кода

Вывод: без пустых строк читать код сложнее

Пустые строки

```
if number % 2 == 0:  
    number_type = "even"  
  
else:  
    number_type = "odd"  
  
print(f"I got {number_type} number")
```

Вывод: с пустыми строками читать код легче

Выражение vs Инструкция

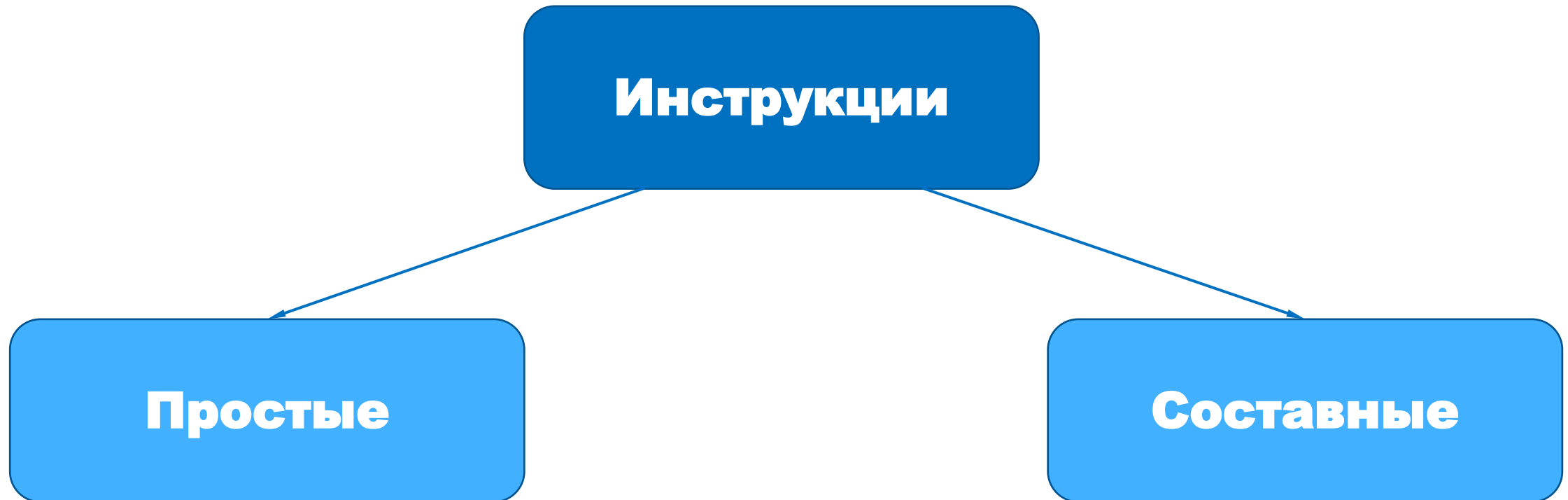
Выражения

```
2 + 2           # 4  
len("Python")   # 6
```

Инструкции

```
num = 10  
import math  
return x
```

Инструкции



Простые инструкции

операция связки - простая инструкция

```
light_velocity = 3e8
```

арифметические операции - простые инструкции

```
2 ** 10
```

вызов функции - простая инструкция

```
print("Hello, world!")
```


Составные инструкции

```
# первый блок сложной инструкции
if number % 2 == 0: # заголовок первого блока
    # тело первого блока
    number_type = "even"

# второй блок сложной инструкции
else: # заголовок второго блока
    # тело второго блока
    number_type = "odd"

# ЭТОТ КОД НЕ ОТНОСИТСЯ К СЛОЖНОЙ ИНСТРУКЦИИ
print(f"I got {number_type} number")
```

Отступы

```
factorial = 1  
print(f"init value: {factorial}")
```

} первый блок кода

```
for i in range(1, 6):  
    factorial *= 1  
    print(f"[ITER_{i}]: {factorial}")
```

} второй блок кода



отступ - 4 пробела

Значение отступов

Вариант 1

```
for i in range(2):  
    print(i)  
    print("---")
```

Вывод 1

```
0  
---  
1  
---
```

Вариант 2

```
for i in range(2):  
    print(i)  
print("---")
```

Вывод 2

```
0  
1  
---
```



Токены

Токенизация

исходный код

```
total = price * 2
```

результат токенизации

total	идентификатор
-------	---------------

=	оператор
---	----------

price	идентификатор
-------	---------------

*	оператор
---	----------

2	число
---	-------

Токены

- **идентификаторы**
- **ключевые слова**
- **литералы**
- **операторы**
- **разделители**

Идентификаторы

<code>my_variable = 5</code>	<code># валидный идентификатор</code>
<code>MyVariable = 6</code>	<code># валидный идентификатор</code>
<code>_var1 = 1</code>	<code># валидный идентификатор</code>
<code>__len__ = 4</code>	<code># валидный идентификатор</code>
<code>_2 = 2</code>	<code># валидный идентификатор</code>
<code>2var = 2</code>	<code># SyntaxError</code>
<code>my_\$uper_variable = 4</code>	<code># SyntaxError</code>

Соглашения об именовании

идентификаторы переменных в snake_case

```
light_velocity = 3e8
```

идентификаторы функций в snake_case

```
def get_filtered_data():
```

```
...
```

идентификаторы модулей в snake_case

```
import numpy as np
```

идентификаторы классов в PascalCase

```
class DataSelector:
```

```
...
```


Ключевые слова

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Мягкие ключевые слова

match

case

type

—

Литералы

1	# целочисленный число
3.14	# литерал числа с плавающей точкой
1.0j	# литерал комплексного числа
"Hello!"	# строковый литерал
[1, 2, 3]	# списковый литерал
{"a": 1, "b": 2}	# словарный литерал

Магические константы

так делать не надо

```
num = get_random_number(seed=42)
```

...

```
num_other = get_random_number(seed=42)
```

лучше делать так

```
seed = 42
```

```
num = get_random_number(seed=seed)
```

...

```
num_other = get_random_number(seed=seed)
```

Операторы

Группа	Токены
Арифметические	+ - * ** / // %
Сравнение	< > <= >= == !=
Битовые	& ^ ~ << >>
Составное присваивание	+= -= *= /= ...
Морж	:=

Разделители

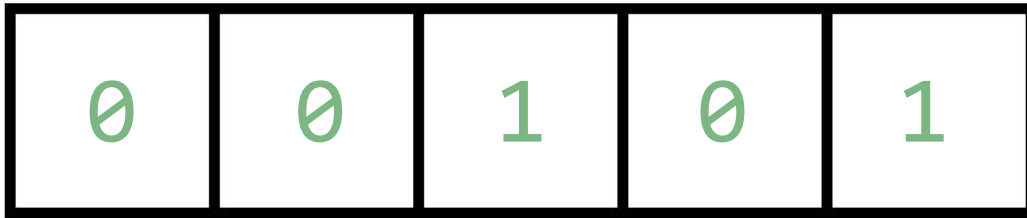
Токены	Роль
() [] { }	скобки
,	разделяет элементы
;	несколько инструкций в строке
:	срезы, словари, тело блока
->	тип возврата



Объекты Python

Объекты

данные в памяти компьютера



данные в python-программе

PyObject

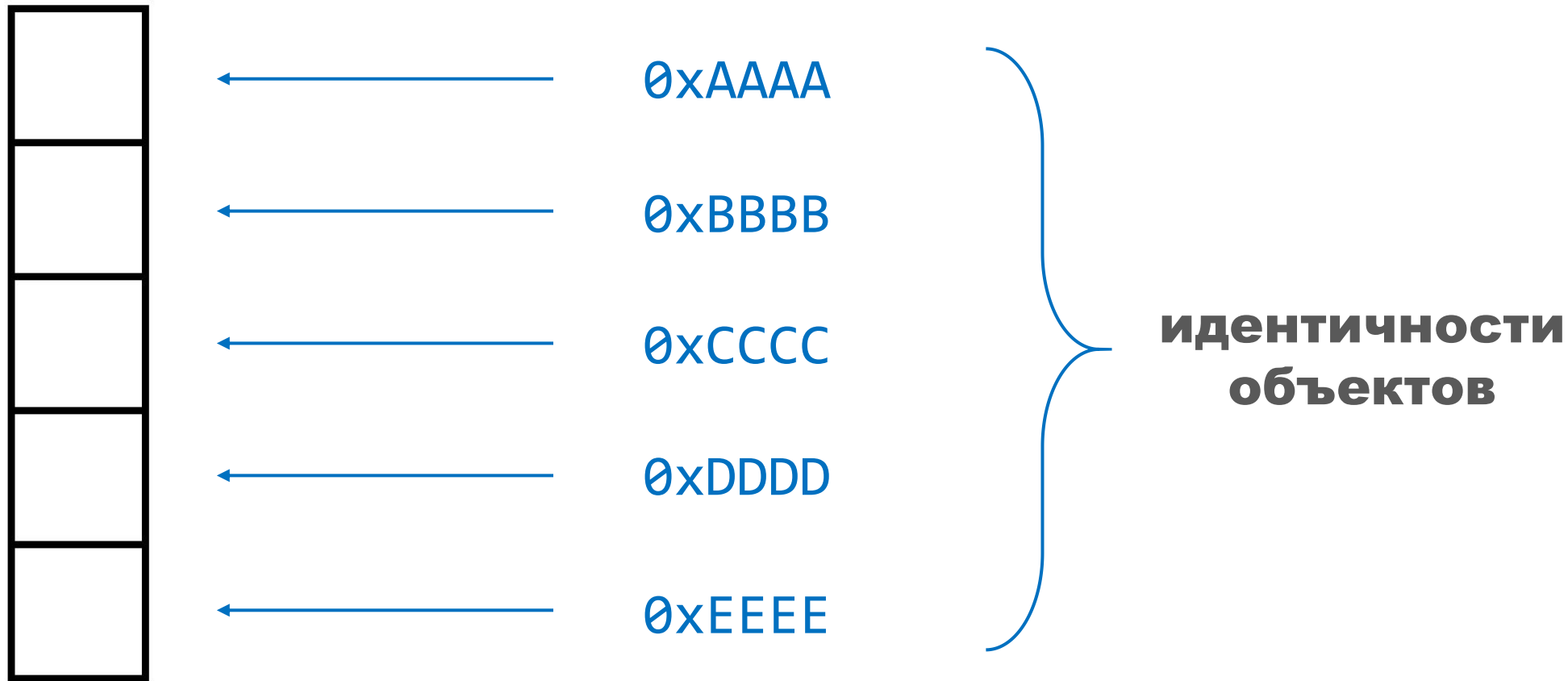
Характеристики объекта

- **идентичность**
- **тип данных**
- **значение**

Идентичность в CPython

ячейки памяти

адреса ячеек



Работа с идентичностью

получение идентичности объекта с помощью id()

```
>>> id(5)
```

```
140723671524264
```

сравнение идентичностей объекта

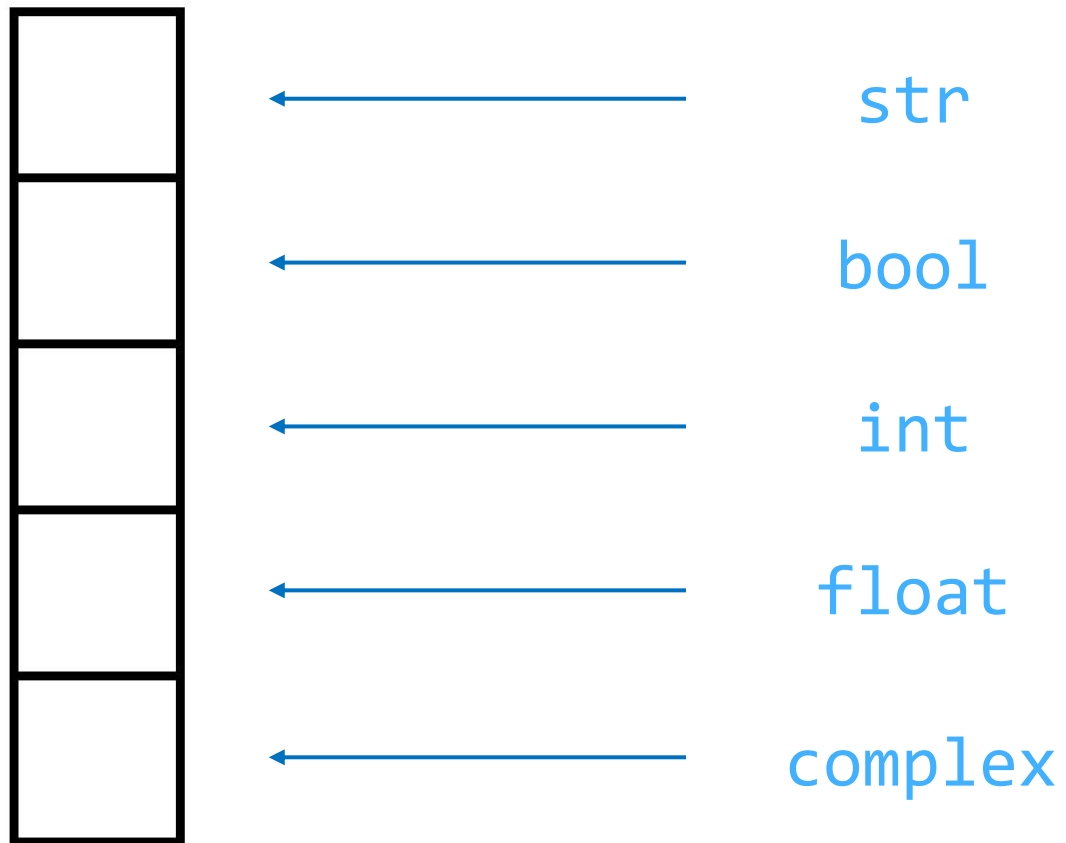
```
>>> print() is None
```

```
True
```

Тип данных

ячейки памяти

тип данных



тип данных определяет:

- **допустимые значения**
- **допустимые операции**

Работа с типом данных

определение типа данных объекта с помощью type

```
>>> type(None)
<class 'NoneType'>
>>> type(None).__name__
'NoneType'
```

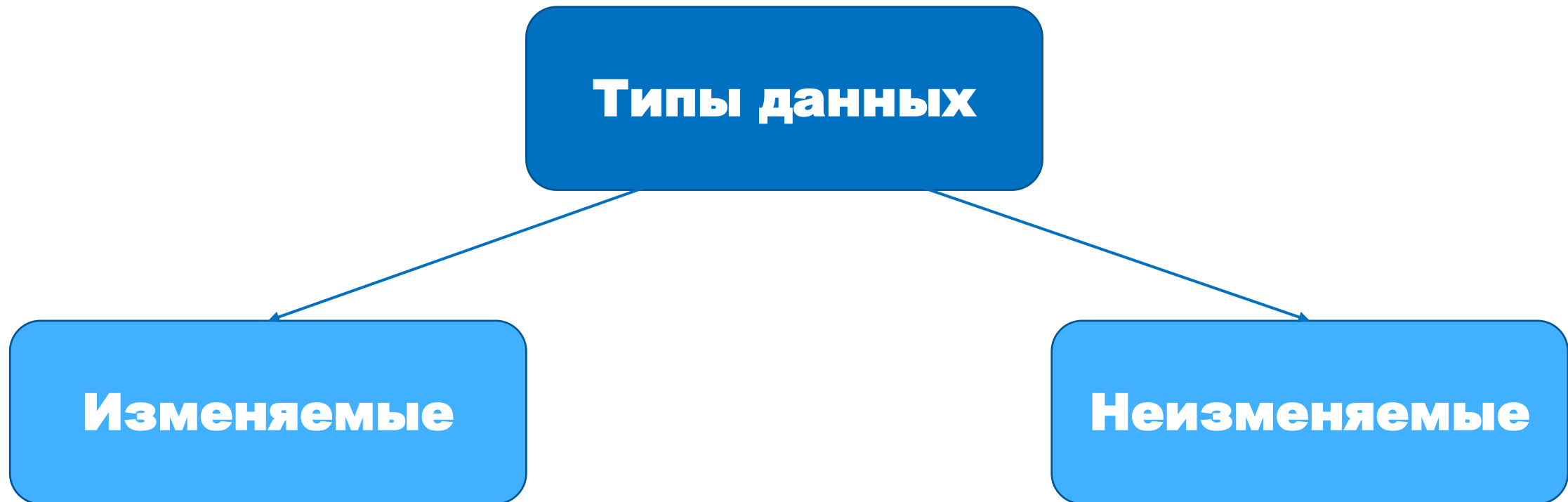
проверка типа данных объекта с помощью type

```
>>> bool_flag = True
>>> type(bool_flag) is bool
True
```

проверка типа данных объекта с помощью isinstance()

```
>>> bool_flag = True
>>> isinstance(bool_flag, bool)
True
```

Разновидности типов данных



Значения объектов

ячейки памяти

тип данных

значения

	←	str	←	"abc"
	←	bool	←	True
	←	int	←	42
	←	float	←	3.14
	←	complex	←	1 + 1j

Переменные

создание переменных с помощью оператора =

```
num1 = 365
```

```
num2 = num1
```

перепривязка переменной с помощью оператора =

```
num1 = "spam"
```


Работа с переменными

num1 = 365

num1

365

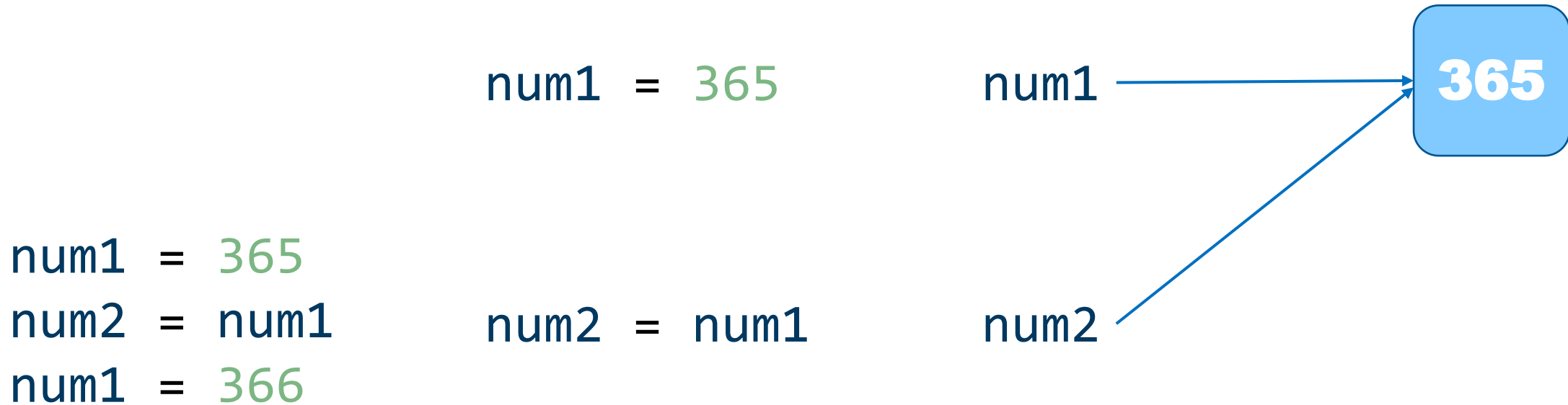
num1 = 365

num2 = num1

num1 = 366

num2 = num1

num2



Работа с переменными

num1 = 365

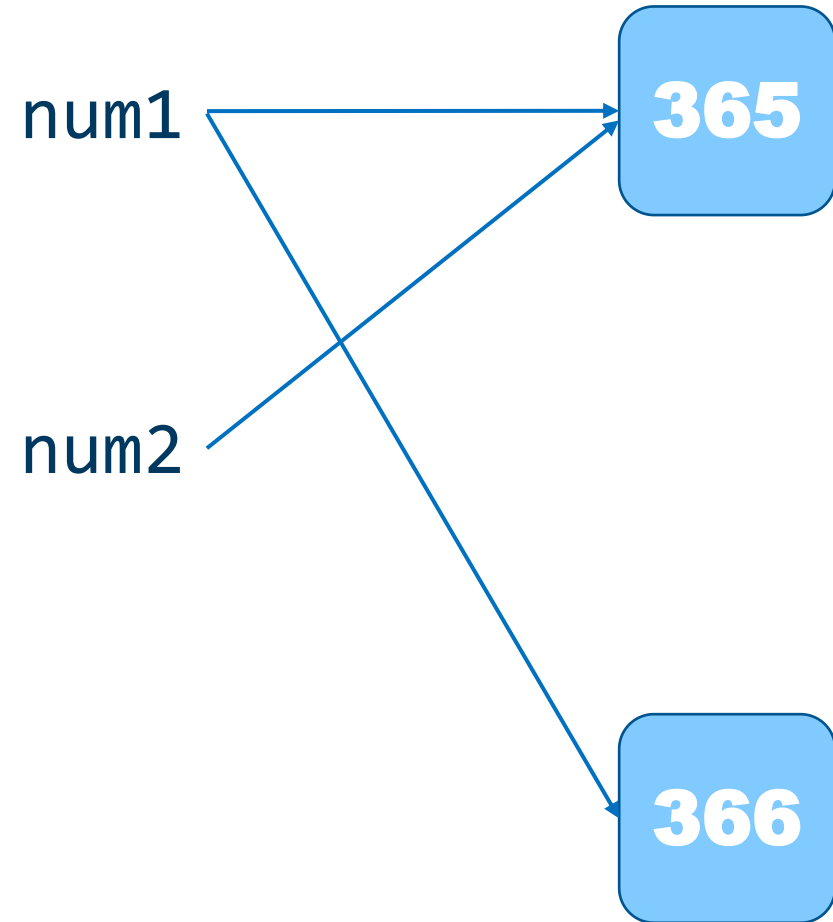
num1 = 365

num2 = num1

num1 = 366

num2 = num1

num1 = 366



Проверка

```
>>> num1 = 365  
>>> num2 = num1  
>>> print(num1 is num2)  
True
```

```
>>> num1 = 366  
>>> print(num1 is num2)  
False
```



Управляющие конструкции

Истинность условия

Ложно (falsy)

`False, None`

`0 0.0 0j`

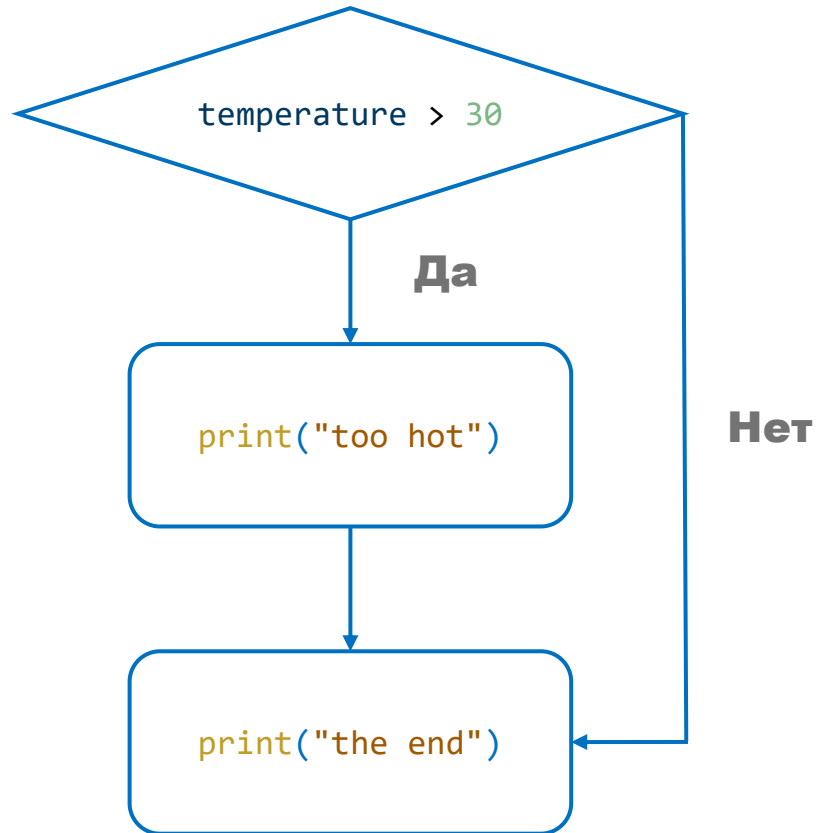
`""`

`[] {} set() ()`

Истинно (truthy)

Все остальное

Ветвление

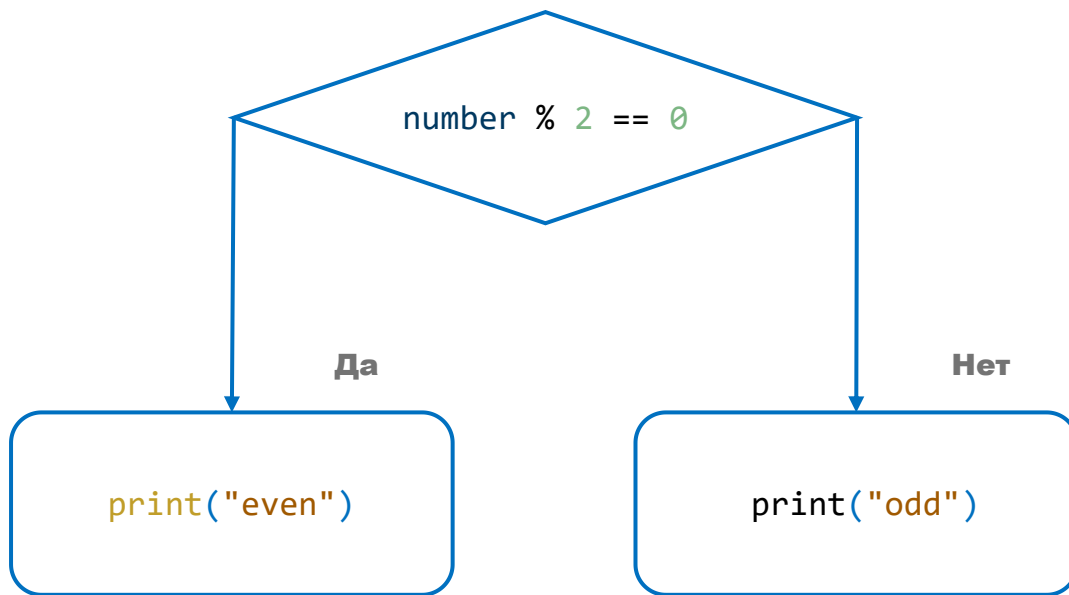


```
temperature = 35
```

```
if temperature > 30:  
    print("too hot")
```

```
print("the end")
```

if-else

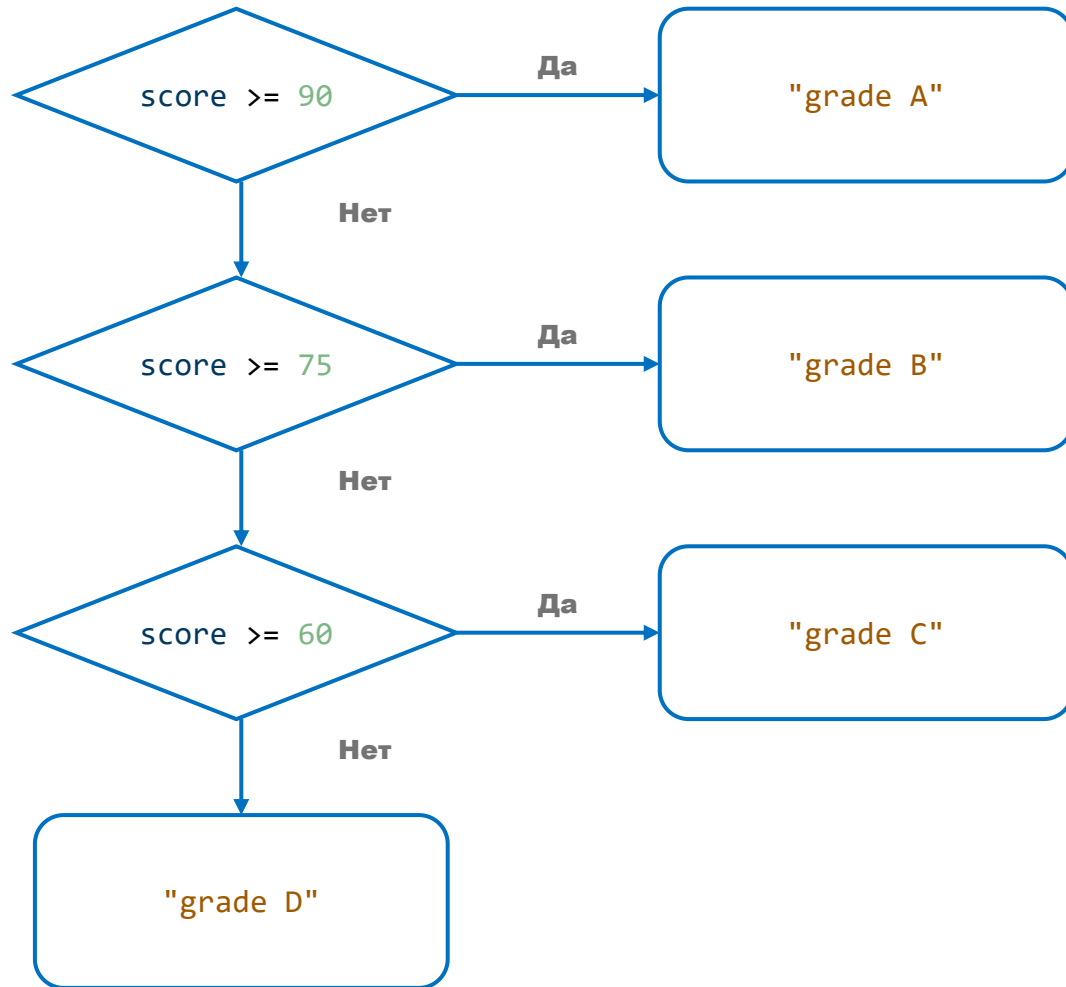


```
number = 7
```

```
if number % 2 == 0:  
    print("even")
```

```
else:  
    print("odd")
```

if-elif-else



score = 78

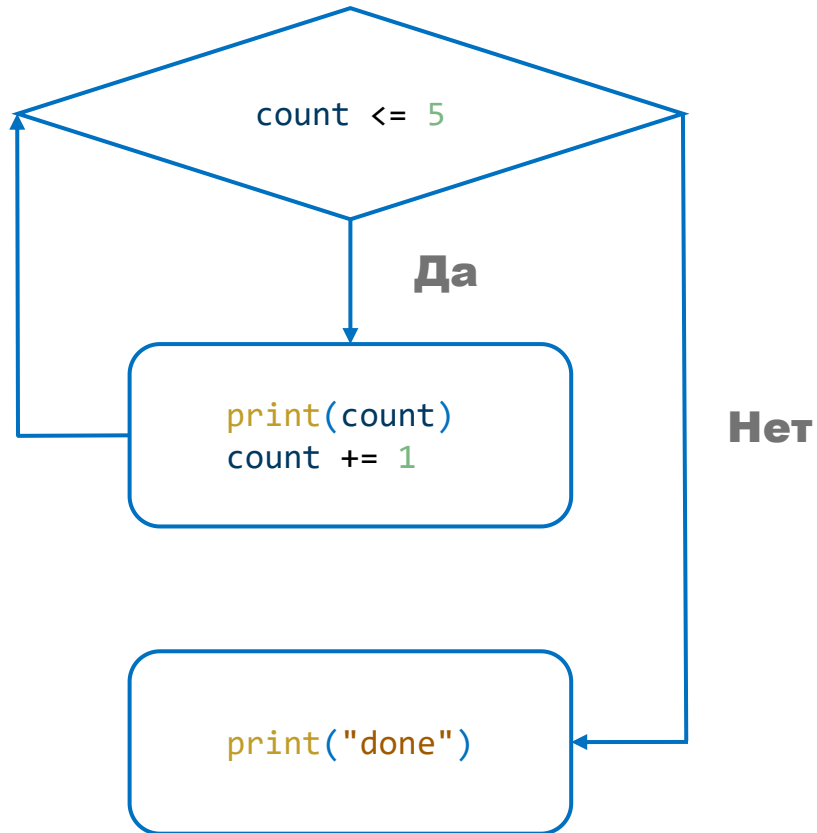
```
if score >= 90:  
    grade = "grade A"
```

```
elif score >= 75:  
    grade = "grade B"
```

```
elif score >= 60:  
    grade = "grade C"
```

```
else:  
    grade = "grade D"
```


Цикл while



```
count = 1
```

```
while count <= 5:  
    print(count)  
    count += 1
```

```
print("done")
```



Спасибо за внимание!